

CONTEXTGRAPH: TESTING WHEN REPAIR EXPERIENCE APPLIES

Anonymous authors

Paper under double-blind review

ABSTRACT

A past repair is useful only under the conditions that made it work. Coding-agent memory commonly retrieves an issue, patch, or rule and leaves the agent to infer these conditions. We develop an applicability-driven alternative: turn retrieved experience into a repair hypothesis, identify an observation that would support or contradict it in the current repository, and execute that check before committing to the suggested edit. The central hypothesis is that memory should reduce uncertainty about the next action, rather than merely increase the amount of relevant text. Our motivating experiments on 99 SWEContextBench Related-Lite tasks find 17 resolved tasks without memory, 18 with flat patch retrieval, and 18 with an added repository procedure card. A training example exposes a concrete ambiguity: a string conversion that fixes an HTTP-method bug under one input and runtime becomes the cause of a different bug under another. We specify the resulting method and the comparisons needed to test it. The method’s repair benefit remains under investigation; the completed results motivate the intervention.

Working draft. The method is being developed on training tasks. The result table below describes completed earlier interventions; it is not a result for the new method.

1 INTRODUCTION

Software repairs encode conditional knowledge. Adding a conversion, changing an ordering constraint, or copying an object can fix a bug because a particular input, runtime, and program state make that change necessary. Removing those conditions turns a useful experience into advice that may point in the wrong direction.

This matters for coding agents because issue similarity is only an indirect guide to the next useful action. Two issues can mention the same API and exception while requiring different edits. A memory that says what worked previously still leaves the agent to determine what is true now. Our starting point is to make that determination part of memory use.

We propose to organize experience around *testable repair hypotheses*. For each retrieved candidate, the agent identifies its applicability condition, a small check that distinguishes competing explanations, and the edit justified by the check’s outcome. The check runs against the current repository before the agent adopts the repair. The result can support the candidate, reject it, or leave it unresolved. Rejected experience remains useful: it eliminates an attractive but unsuitable direction.

The proposed contribution is this connection between experience retrieval and active diagnosis. Graph retrieval supplies candidate experiences; execution determines how they should influence the current decision. The paper asks whether this connection improves repair success, whether its benefit comes from executing the condition checks, and whether it helps when superficially related experiences prescribe incompatible actions.

2 WHY RELEVANT EXPERIENCE CAN BE INSUFFICIENT

More guidance did not produce a clear gain. Our completed exploratory Related-Lite experiment compares six memory interfaces on the same 99 tasks from SWEContextBench (Zhu et al., 2026). The solver is SWE-agent 1.1.0 with a DeepSeek V4 Pro model alias, temperature zero, a

Table 1: Completed Related-Lite results used to motivate method development. Each condition uses all 99 tasks and the benchmark’s frozen verifier. These are earlier interventions, separate from the new applicability method.

Memory interface	Resolved	Difference from no memory
No memory	17/99	—
Flat patches	18/99	+1.01 pp
Patches + procedure card	18/99	+1.01 pp
Graph	12/99	−5.05 pp
Rules	13/99	−4.04 pp
File list	9/99	−8.08 pp

500-call limit, and a history processor retaining the last five observations. Flat retrieves historical patches; Card adds a repository procedure card; Graph, Rules, and File list expose other representations of the available experience. Table 1 retains every original condition.

Flat and Card each gain one observed repair over no memory; both exact paired tests give $p = 1$. Flat and Card share 12 successes and have six unique successes each. Their paired difference is 0 pp, with a 95% bootstrap interval of $[-7.07, 7.07]$ pp. Thus richer repository guidance did not produce a demonstrated gain in this setting.

Patch memory reaches all 99 targets in both conditions, and the expected procedure card reaches 98. However, reconstructing 198 Flat/Card histories shows that an original memory response is retained for a median of only five action prefixes. The median trajectories contain 497 and 495 actions, respectively. This identifies a delivery constraint, while leaving open the more consequential question of what the agent should do with the experience. Our new comparisons keep memory persistent in every condition so that retention is held fixed.

The same operation can be a repair or a bug. A Requests training example makes the applicability problem concrete. Issue 1724’s patch adds `method = builtin_str(method)` to normalize an HTTP method in a Python 2 setting. Issue 2317 instead reports that this conversion turns Python 3 bytes `b'GET'` into the literal string `"b'GET'"`. Its training patch uses `to_native_string(method)`. A retrieved recommendation to perform the older conversion is related to the target but does not settle the correct action.

A short execution can distinguish the two situations: inspect the input type and compare the converted value with the intended method name. The transferable lesson is the condition on the conversion, including the case in which it should be rejected. This inspected training example motivates the method; it does not estimate how frequently such conflicts occur.

3 APPLICABILITY-DRIVEN EXPERIENCE USE

Let s_t contain the issue, current code, runtime, and observations available at step t . An experience contains a source problem and its repair evidence. Instead of mapping issue text directly to advice, the proposed procedure follows

$s_t \longrightarrow$ candidate experiences $\longrightarrow$ discriminating check $\longrightarrow$ observation $\longrightarrow$ next repair action.

Retrieve candidates. ContextGraph retrieves a small shared pool through dense similarity, lexical matching, and Personalized PageRank over experience and mechanism nodes. Reciprocal-rank fusion and diversity reranking produce the candidate packet. For the first method experiment, this packet is computed once and held identical between conditions. This isolates how experience is used after retrieval.

Make the repair hypothesis explicit. For candidate m , the agent forms

$$h_m = (c_m, q_m, e_m),$$

where c_m states the conditions needed by the source repair, q_m is a check of a condition that matters for the target, and e_m is the edit or debugging action supported by that condition. The check must

distinguish plausible actions. Printing a version number is useful only when the version changes whether the repair applies; rerunning an unrelated passing test does not distinguish the candidates.

The initial implementation uses a common solver and tools to construct these hypotheses from the retrieved packet and current code. It does not require a separate model or a larger memory bank. At most two short checks are requested before editing, so the procedure spends its diagnostic budget on the most consequential uncertainty.

Execute, then choose. A check returns supporting evidence, contradicting evidence, or an unresolved result. The agent records the observation together with the candidate it concerns. Supporting evidence allows the agent to adapt the proposed action to the target. Contradicting evidence removes that candidate from the current repair decision. An unresolved check, including an environment error, does not decide applicability. If the retrieved candidates remain unhelpful, the solver continues from the issue and repository.

This three-way decision avoids treating an unsuccessful check as evidence that a repair is wrong. It also makes a negative memory result actionable: the agent can stop repeating a tempting edit whose required condition is absent.

Validate the resulting repair. The solver reruns its issue reproduction after editing and checks relevant existing tests. The benchmark verifier grades each submitted patch immediately after that task. The method’s local condition checks use information available to the solver; the benchmark’s held-out tests remain evaluation targets. At evaluation time the source memory bank is fixed.

4 EXPERIMENTS THAT TEST THE ARGUMENT

The development sequence centers on three questions. Each comparison changes a component needed by the method’s argument.

Does checking applicability improve repairs? Compare the same persistent memory packet with ordinary use and with explicit applicability checks. Keep the model, tools, task images, generation settings, and total budget equal. The outcome is verified repair success; trajectories show whether a candidate was accepted or rejected before editing. The first local pilot uses training tasks and a deliberately small shared budget to determine whether the intervention produces the intended behavior.

Does execution supply information beyond more reasoning? Compare executed checks with a condition that writes down the same hypotheses and expected outcomes but does not run the extra checks. All conditions retain the ordinary issue reproduction and final testing. This comparison asks whether observing the current program changes repair decisions, rather than attributing a gain to additional instructions alone.

Do applicability boundaries explain transfer? Examine development cases in which related experiences recommend incompatible actions, alongside cases where the source repair applies directly. For each case, record the candidate decision before and after the discriminating observation and the resulting patch. A useful mechanism result is a wrong candidate being rejected and a correct repair following from that observation. Final selection and reporting must include the cases where probing costs time or rejects useful experience.

Evaluation split. The separate SWE-bench Verified campaign (Jimenez et al., 2024) allocates 350 tasks to memory construction and development and 150 to evaluation, with repository-stratified allocation and seed 42. Within the 350-task pool, an existing 70-task development partition excludes all of its source rules from the remaining 280-source bank. The bank contains 376 rules. These development tasks support method selection without using the 150 evaluation tasks. The earlier 150-task experiment is incomplete and supplies no aggregate effect estimate for this draft.

The principal evaluation will compare the selected method against no memory and the same memory without applicability checks. The decision to retain graph retrieval will depend on its contribution

with memory use held constant. A larger graph, additional prose, or a new audit is not itself an experimental objective.

5 RELATED WORK

Experience and skills. ExpeL distills reusable insights from agent trajectories (Zhao et al., 2024). SWE-Exp uses historical experience for software issue resolution (Chen et al., 2025), and MemGovern turns human experience into searchable cards (Wang et al., 2026). CODESKILL learns a skill extraction and maintenance policy using feedback from a frozen downstream solver (Li et al., 2026). These approaches establish experience as a useful object of learning. Our question concerns how a retrieved experience should be tested against the current program before it determines an action.

Repair patterns and context. Getafix learns fix patterns from human patches and uses surrounding code to rank applicable edits (Bader et al., 2019). Thus executable edits and context-sensitive repair reuse are established ideas. The proposed direction here is interactive discrimination: an agent uses a retrieved experience to choose an observation, and that observation changes which repair hypothesis it follows. Its value must be established against ordinary repair reasoning and testing.

Graph retrieval and working memory. HippoRAG motivates associative retrieval with Personalized PageRank (Gutiérrez et al., 2024). SWE-MeM studies working-memory management (Gao et al., 2026), while cue-anchored memory studies delivery through coding-task and compaction experiments (Saha, 2026). These address candidate access and retention. Holding delivery fixed lets our experiments focus on the information gained by checking applicability.

6 SCOPE OF THE CURRENT EVIDENCE

The completed Related-Lite results are exploratory and include related source fixes that can closely match target repairs. The benchmark’s frozen *resolved* criterion requires fail-to-pass success but does not veto every pass-to-pass regression. We therefore use those results to motivate development, while reporting new Verified results under its own harness. The Requests example is an inspected training case. Neither it nor the earlier aggregate scores establishes a repair gain for the proposed method.

The unresolved scientific question is whether experience-guided checks choose better actions often enough to repay their cost. A successful method should improve verified repairs at a matched budget and show decisions changed by informative observations. If the checks merely lengthen trajectories, the intervention has not achieved its purpose.

REPRODUCIBILITY STATEMENT

Code, task splits, trajectories, patches, and per-task verification records accompany the working repository. The completed result table can be recomputed from the existing statistical package. Operational logs and detailed evidence inventories are maintained with the experiments.

AI USE STATEMENT

Generative AI assisted literature review, method development, implementation, experiments, analysis, and writing. The manuscript remains a working draft pending human review.

REFERENCES

- Johannes Bader, Andrew Scott, Michael Pradel, and Satish Chandra. Getafix: Learning to fix bugs automatically, 2019. URL <https://arxiv.org/abs/1902.06111>.
- Silin Chen, Shaoxin Lin, Yuling Shi, Heng Lian, Xiaodong Gu, Longfei Yun, Dong Chen, Lin Cao, Jiyang Liu, Nu Xia, and Qianxiang Wang. SWE-Exp: Experience-driven software issue resolution, 2025. URL <https://arxiv.org/abs/2507.23361>.

- Shuzheng Gao, Wenhao Zeng, Zhaojian Yu, Jianqiao Wangni, Chaozheng Wang, Kai Cai, Shilin He, and Michael R. Lyu. Swe-mem: Learning adaptive memory management for long-horizon coding agents, 2026. URL <https://arxiv.org/abs/2606.28434v1>. Version 1.
- Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. Hipporag: Neurobiologically inspired long-term memory for large language models, 2024. URL <https://arxiv.org/abs/2405.14831v3>.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues?, 2024. URL <https://arxiv.org/abs/2310.06770>.
- Yanzhou Li, Yiran Zhang, Xiaoyu Zhang, Xiaoxia Liu, and Yang Liu. Codeskill: Learning self-evolving skills for coding agents, 2026. URL <https://arxiv.org/abs/2605.25430v1>. Version 1.
- Swapnanil Saha. Delivery, not storage: Cue-anchored working memory as a harness property for coding agents, 2026. URL <https://arxiv.org/abs/2607.20972v1>. Version 1.
- Qihao Wang, Ziming Cheng, Shuo Zhang, Fan Liu, Rui Xu, et al. MemGovern: Enhancing code agents through learning from governed human experiences, 2026. URL <https://arxiv.org/abs/2601.06789v2>.
- Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. Expel: Llm agents are experiential learners, 2024. URL <https://arxiv.org/abs/2308.10144v3>.
- Jiayuan Zhu, Junde Wu, Minhao Hu, Shengda Zhu, Jiazhen Pan, Weixiang Shen, Yijun Yang, Fenglin Liu, Jianye Hao, Yueming Jin, Qirong Ho, and Min Xu. Swe context bench: A benchmark for context learning in coding, 2026. URL <https://arxiv.org/abs/2602.08316v3>. Version 3.